

Universidad Tecnológica Nacional



TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Carrera: Tecnicatura Universitaria en Programación

Materia: Programación 1

Alumna: Olmos, Agustina Celia

Docente: Matias Santiago Torres

Fecha de entrega: 17 / 06 / 2026

Índice

Índice.....	2
1. Introducción.....	3
2. Objetivo del Proyecto	3
3. Marco Teórico	3
3.1 Listas	3
3.2 Diccionarios	3
3.3 Funciones	4
3.4 Condicionales	4
3.5 Estructuras Repetitivas	4
3.6 Ordenamientos	4
3.7 Estadísticas Básicas	5
3.8 Archivos CSV.....	5
4. Decisiones Técnicas y Arquitectura	5
5. Descripción del Funcionamiento del Sistema	6
6. Dificultades Encontradas.....	6
7. Conclusiones.....	6
8. Bibliografía / Webgrafía	6
9. Enlaces al Repositorio y Video.....	6

1. Introducción

En el presente Trabajo Práctico Integrador se desarrolla una aplicación en Python orientada a la gestión de información de países, utilizando estructuras de datos fundamentales como listas y diccionarios. El sistema permite almacenar, consultar, filtrar, ordenar y analizar información proveniente de un archivo CSV, simulando un caso real de procesamiento de datos.

Este proyecto tiene como finalidad integrar los contenidos vistos a lo largo de la materia Programación 1, aplicándolos de manera práctica y organizada.

2. Objetivo del Proyecto

El objetivo principal del proyecto es desarrollar una aplicación en Python que permita:

- Gestionar datos de países.
- Aplicar listas y diccionarios de forma eficiente.
- Implementar funciones para modularizar el código.
- Realizar filtros, ordenamientos y estadísticas.
- Leer y escribir información desde archivos CSV.
- Validar datos y manejar errores básicos.

3. Marco Teórico

3.1 Listas

Las listas en Python son estructuras de datos que permiten almacenar múltiples elementos en una sola variable. Son mutables, ordenadas y permiten acceso por índice.

En este proyecto, se utiliza una lista para almacenar todos los países cargados en el sistema.

Ejemplo conceptual:

```
países = []
```

3.2 Diccionarios

Los diccionarios permiten almacenar información en pares clave-valor. Cada país se representa mediante un diccionario con claves como nombre, población, superficie y continente.

Ejemplo conceptual:

```
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América"
}
```

3.3 Funciones

Las funciones permiten dividir el programa en bloques reutilizables y organizados, facilitando la lectura, el mantenimiento y la corrección del código.

En el sistema, cada funcionalidad (agregar, buscar, ordenar, etc.) está implementada mediante una función independiente.

3.4 Condicionales

Las estructuras condicionales permiten ejecutar diferentes acciones según se cumpla o no una condición determinada.

Se utilizan para:

- Validar opciones del menú.
- Controlar búsquedas.
- Manejar errores de entrada.

Ejemplo:

```
if opcion == "1":
    agregar_pais()
```

3.5 Estructuras Repetitivas

Los bucles permiten repetir acciones mientras se cumpla una condición.

Se utilizan para:

- Mostrar el menú continuamente.
- Recorrer listas de países.
- Calcular estadísticas.

Ejemplo:

```
for pais in paises:
    print(pais)
```

3.6 Ordenamientos

El ordenamiento permite reorganizar los datos según un criterio determinado. En Python se utiliza el método `sort()` junto con funciones `lambda`.

El sistema permite ordenar países por:

- Nombre
- Población
- Superficie

3.7 Estadísticas Básicas

Las estadísticas permiten obtener información relevante del conjunto de datos, como promedios, valores máximos y mínimos.

En el proyecto se implementan:

- País con mayor población
- País con menor población
- Promedio de población
- Promedio de superficie
- Cantidad de países por continente

3.8 Archivos CSV

Un archivo CSV (Comma Separated Values) permite almacenar datos estructurados en formato de texto. Python ofrece el módulo `csv` para su lectura y escritura.

El sistema carga los países desde un archivo CSV al iniciar y guarda los cambios al finalizar el programa.

4. Decisiones Técnicas y Arquitectura

Se optó por una arquitectura simple basada en:

- Un archivo principal (`main.py`)
- Un archivo CSV para persistencia de datos
- Uso de funciones independientes para cada acción
- Menú interactivo en consola

Diagrama de flujo (descripción textual):

Inicio → Cargar CSV → Mostrar menú → Ejecutar opción seleccionada → Volver al menú → Guardar CSV → Fin

5. Descripción del Funcionamiento del Sistema

El programa comienza leyendo los datos desde el archivo CSV. Luego muestra un menú en consola que permite al usuario seleccionar distintas opciones, como agregar países, buscar información, aplicar filtros, ordenar datos o visualizar estadísticas.

Cada opción ejecuta una función específica y, al salir del programa, los datos se guardan nuevamente en el archivo CSV.

6. Dificultades Encontradas

Durante el desarrollo del proyecto se presentaron algunas dificultades, tales como:

- Manejo de errores al leer el archivo CSV.
- Validación de entradas numéricas.
- Organización del código para evitar repeticiones.

Estas dificultades fueron resueltas mediante el uso de validaciones y estructuras condicionales.

7. Conclusiones

Este Trabajo Práctico Integrador permitió aplicar de manera práctica los conceptos fundamentales de Programación 1. El desarrollo del sistema fortaleció la comprensión de estructuras de datos, funciones y manejo de archivos, además de fomentar la resolución de problemas de forma autónoma.

8. Bibliografía / Webgrafía

Material otorgado por la catedra

9. Enlaces al Repositorio y Video

Repositorio GitHub del proyecto: <https://github.com/agustinaceliaolmos-boop/Trabajo-practico-integrador.git>

Video demostrativo del sistema: <https://youtu.be/jZY8ohEhL2g>
